

TARGET-STATE ARCHITECTURE

Enact: a work system for **people and agents**

Business, application and technology architecture for enterprise evaluation of human-agent delivery.



What this deck covers

01 The case for a work system

Why coding agents create a coordination problem, and what Enact claims.

03 Application architecture

Components, interaction patterns, integrations and extension surfaces.

02 Business architecture

Capabilities, roles, the value chain, and the commercial model.

04 Technology architecture

Stack, deployment, the data boundary, security and reliability.

01 / 04

THE PROBLEM

Agents got faster.
Coordination did not.

THE PROBLEM

Four gaps that appear after agent adoption

Context is lost

Sessions forget everything when they end. Background is pasted in again and again; one piece of work is scattered across terminals and chat logs.

Accountability is fuzzy

Who started this agent, what may it read, where may it write, and on whose behalf is it acting?

Cost is invisible

Token and compute spend sits in individual accounts and cannot be attributed to a project or a team.

Know-how does not accumulate

What works lives in one person's prompts. A new person, or a new project, starts over.

None of these is a model capability problem. All four are a **missing work system**.

POSITIONING

What Enact is — and what it is not

Enact claims the layer between the model and the plan: the facts, accountability and evidence of enterprise work.

NOT THIS

Two adjacent categories Enact deliberately does not enter.

- No in-house model and no in-house coding agent — reasoning is rented from external runtimes
- Not another plan-of-record tool — the board is one projection, not the only entry point
- No promise of a filesystem sandbox it cannot actually enforce

The model layer is commoditising fast.

THIS

The layer that gets more valuable as general agents get stronger.

- Agents as first-class assignees with identity, charter and visibility scope
- One record, one timeline and one audit trail shared by people and agents
- Capability captured as reusable assets rather than individual prompts

23 supported agent CLI protocol families.

Eight architecture principles

01

Agents are first-class assignees

Assignable, accountable, able to move status.

02

Identity is separate from compute

The agent is who; the runtime is where.

03

The data boundary is fixed

Identical on cloud and self-hosted.

04

Humans authorise the gates

A model may never declare a gate passed.

05

Server and client state stay apart

One source of truth per class of state.

06

Every run leaves evidence

An accountable person and a full record.

07

Extension before built-ins

Five surfaces instead of welded-in logic.

08

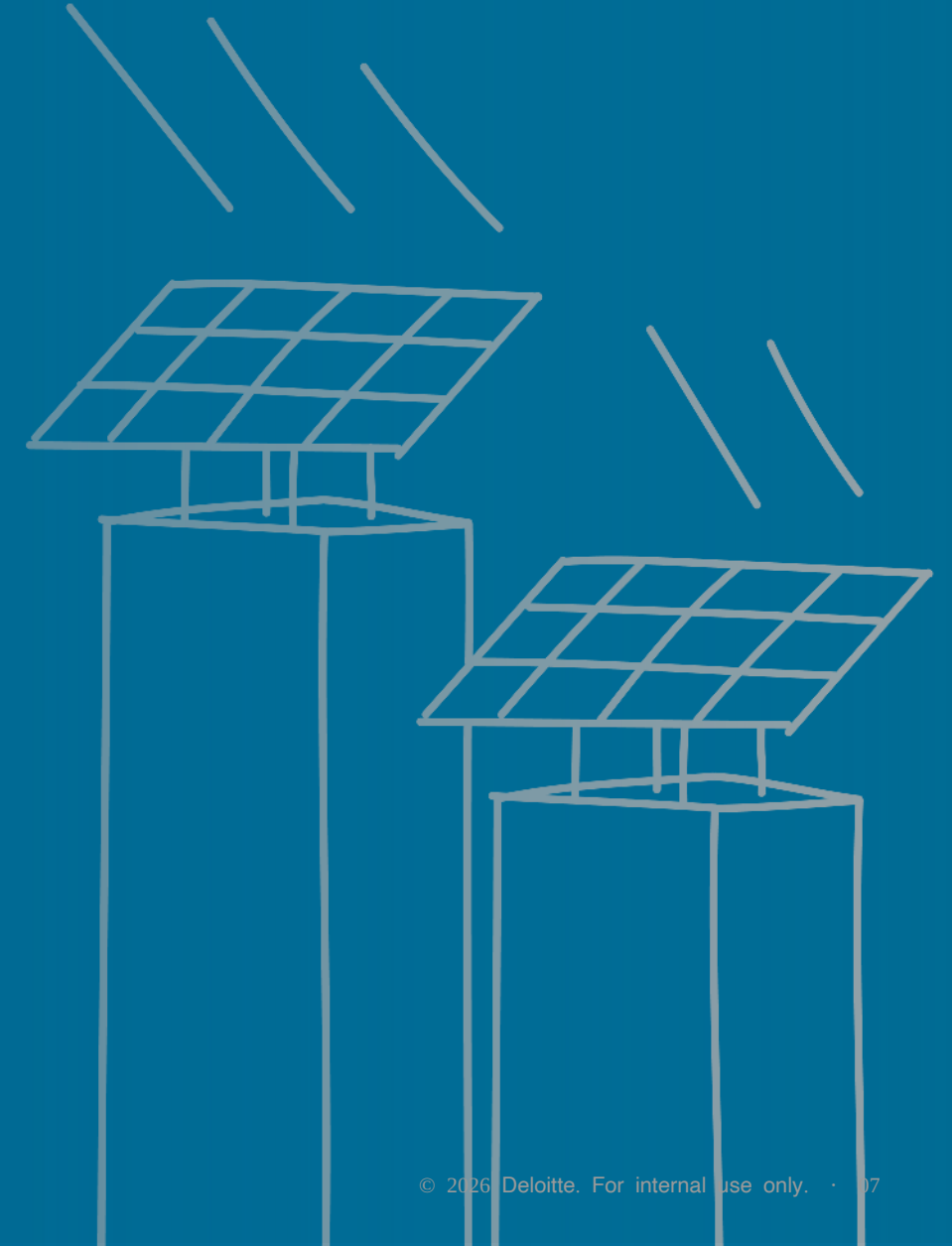
Optional subsystems degrade

Local faults never escalate to outages.

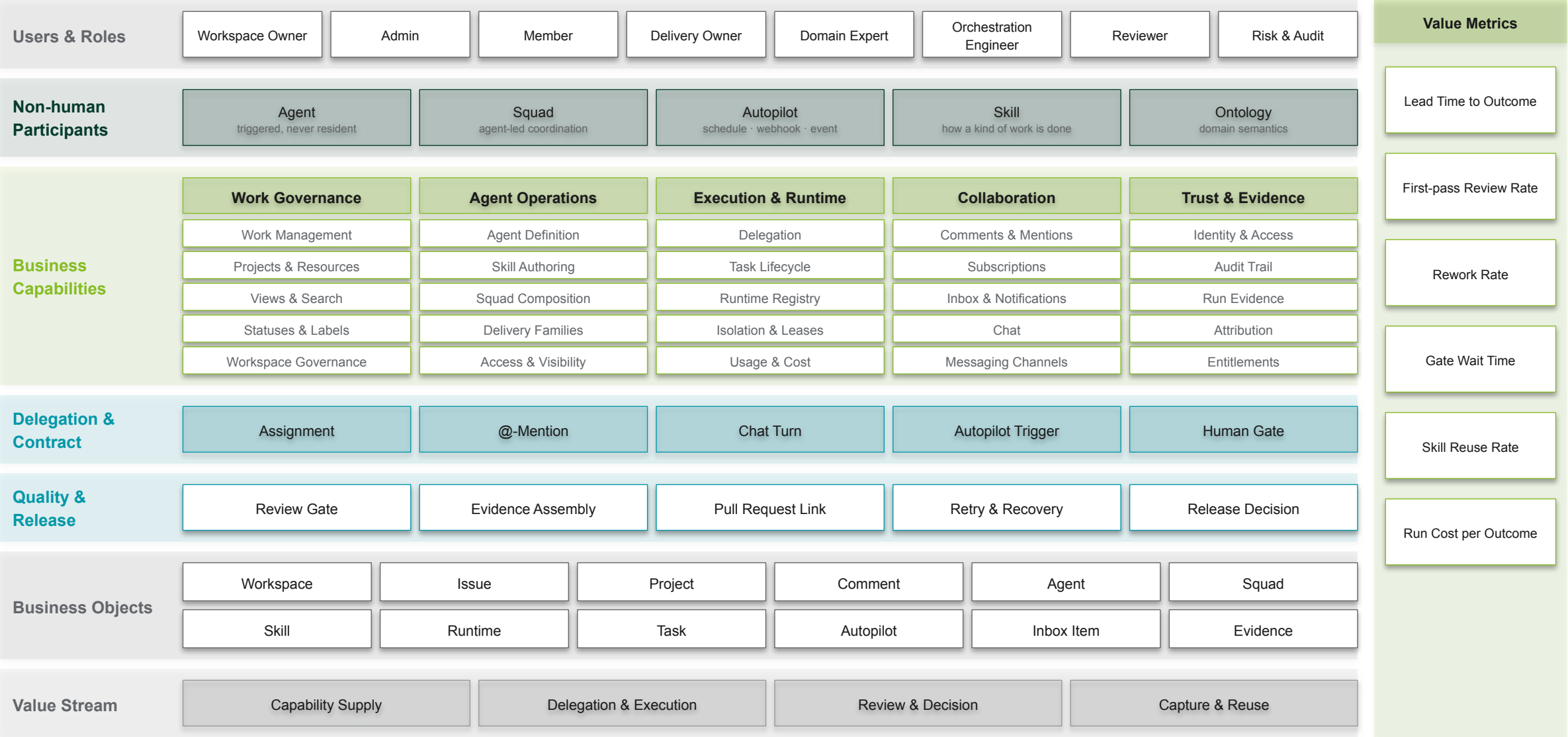
02 / 04

BUSINESS ARCHITECTURE

What the platform does, and who is accountable



Business Architecture



ROLES

Who is accountable, and who may execute

ORGANISATION ROLES

owner	Ownership, billing and subscription, irreversible actions
admin	Members and invitations, integrations, settings, plugin enablement
member	Everyday work: creating and advancing work, configuring agents

Organisation roles govern settings and team management only — they are **not** the same thing as resource visibility.

NON-HUMAN PARTICIPANTS

Agent	A reusable configuration — not a long-running process. Executes only when triggered.
Squad	Agents and members led by an agent leader, which coordinates the work.
Autopilot	Triggers runs on a schedule or from an external event.

ACCESS — PER-RESOURCE VISIBILITY

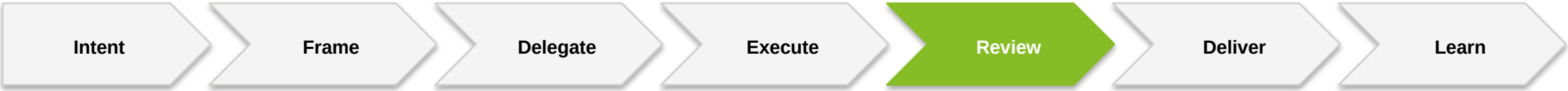
Only me
default

Entire workspace

Specific people
allowlist

Admins can **manage** a resource but **cannot bypass** Access to use it. Management rights and usage rights are separate.

From intent to captured capability



Two links carry the weight: **review is a human judgment** — the model only gets the evidence in order — and **learning flows back** into the next delegation.

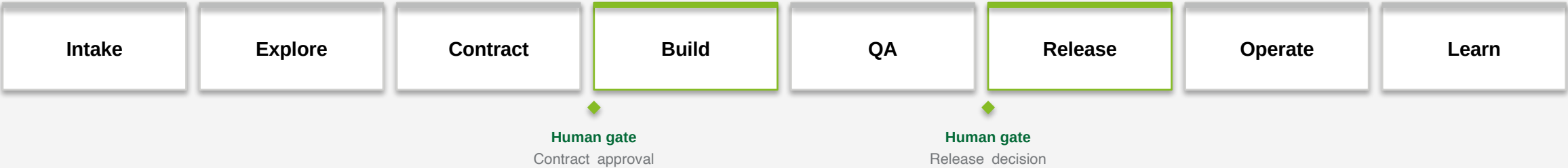
FOUR PEER WAYS TO DELEGATE

Assign work Formal work with a defined deliverable that needs a record Set the assignee to an agent or squad	@-mention Pull an agent into an ongoing discussion for one specific point Mention the agent in a comment	Chat Questions, exploration and quick experiments Each message triggers one run	Autopilot Recurring work, or work driven by external events Schedule, webhook or system event
---	---	--	--

One rule spans all four: **agents never start work on their own**. Every run originates in an explicit action.

AI-SDLC: a governed delivery lifecycle

Every new workspace is provisioned with nine system agents and a delivery squad led by an orchestrator.



Explore before build

Establish goals, boundaries and authoritative sources first — rather than dumping context on a model.

Verification is independent

Test intent derives from the approved contract, not reverse-engineered from the implementation.

Humans authorise the gates

Contract approval and the release decision are human nodes. The model may not declare them passed.

The same skeleton is already instantiated outside software delivery — a **marketing-mix-modelling delivery family** uses the identical structure.

COMMERCIAL MODEL

Open core, seats that exclude agents, metered compute

LICENSING

Internal self-hosted use	No commercial licence
Backend / daemon / CLI only	No licence; attribution retained
Hosted service to third parties	Commercial licence required
Embedded in a distributed product	Commercial licence required
Removing Enact branding	Separate branding waiver

In practice: use it yourself, free; resell it, licence it.

METERING

Workspace subscription	HUMAN SEAT
Agents never consume a seat. Seats cover human members and pending invitations.	
Credit wallet	PREPAID CREDITS
Meters managed cloud runtime compute and model gateway calls.	
Self-hosted	NOT METERED
Self-hosted deployments are unmetered by default.	



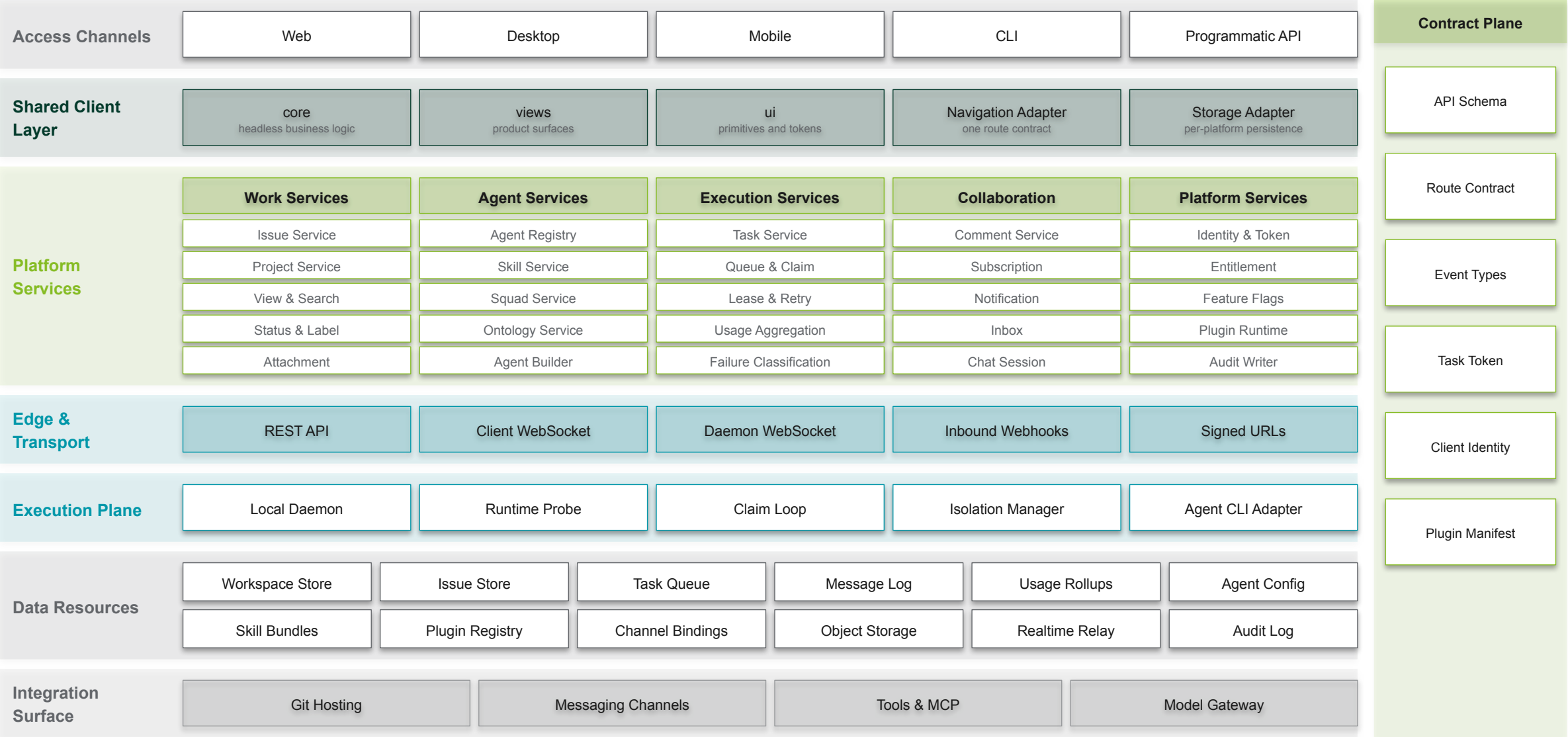
03 / 04

APPLICATION ARCHITECTURE

How the system decomposes, and where it extends



Application Architecture



Clients, shared layer, service domains

CLIENTS	SHARED LAYER	SERVICE DOMAINS
Web Full browser workspace; hosts marketing and docs entry Desktop Tabs, detached windows, native notifications, auto-update, bundled CLI and daemon panel Mobile Independently implemented; shares only types and pure functions CLI Full command surface — and the daemon binary itself	core API client, server state, client state, route contract, permissions, i18n views All product pages and components ui Primitives and design tokens Three adapters Navigation, storage and runtime configuration — the only platform seam	Work · Agent Work lifecycle, projects, views; agents, skills, squads, ontologies Execution Queue, claim, lease, streaming progress, retries, usage Collaboration Comments, subscriptions, notifications, inbox, chat Integration · Platform Git, channels, MCP; identity, plugins, entitlements, flags

END TO END

From assignment to pull request

Task dispatch is pull-based: the server never dials out to an execution machine, so it can sit inside the corporate network with no inbound ports.

- | | | | |
|----|--|----|---|
| 01 | DELIVERY OWNER
Assigns the work to an agent | 06 | AI CODING TOOL
Runs as a subprocess with a task-scoped credential |
| 02 | APPLICATION SERVICES
Resolves agent config, assembles context and skills | 07 | AI CODING TOOL
Streams progress back throughout the run |
| 03 | TASK QUEUE
Task enqueued and attributed to the initiator | 08 | GIT HOST
Pushes a branch and opens a pull request |
| 04 | LOCAL DAEMON
Claims the task and takes a lease | 09 | APPLICATION SERVICES
Writes back to the timeline, aggregates cost, notifies |
| 05 | LOCAL DAEMON
Prepares an isolated execution environment | 10 | DELIVERY OWNER
Reviews the evidence and decides |

■ Human ■ Enact control plane ■ Your machines

A completed run is not completed work. One issue can carry many runs; a human decides to merge, reject or continue.

Meeting the enterprise where it already works

Code hosting

- GitHub App — installed per workspace
- Self-hosted Git — GitLab, Gitea, Forgejo
- Signature-verified inbound webhooks
- Pull requests, check runs and suites tracked

Enact does not host code. It records the link between work and pull requests.

Corporate messaging

- Slack · Lark · WeCom
- DingTalk · Telegram
- One unified channel engine, not five
- Account binding maps identity to a member

Reach agents from the tools the team already uses — without relaxing permissions.

Tools and capability

- Composio connected accounts
- MCP servers — workspace, agent, plugin
- Replaceable model gateway endpoint
- Object storage, email, payments

Remote tool credentials resolve per connection and never enter the task record.

Five surfaces, so the core stays the core

SURFACE	EXTENDS	AUTHOR	GOVERNANCE
skill	How an agent does a kind of work	Domain expert	Authored or imported; attaches to many agents; bound by Access
Ontology	Domain semantics and terminology	Domain expert	Attached to agents as a domain concept baseline
Runtime Profile	Which machine and which tool executes	Orchestration engineer	Per workspace; only on officially supported protocol families
MCP Server	What tools an agent can call	Engineer / admin	Workspace, agent or plugin-bridged; credentials on demand
Plugin	The Enact platform itself	Third-party developer	Immutable versions; installation and grants need admin consent

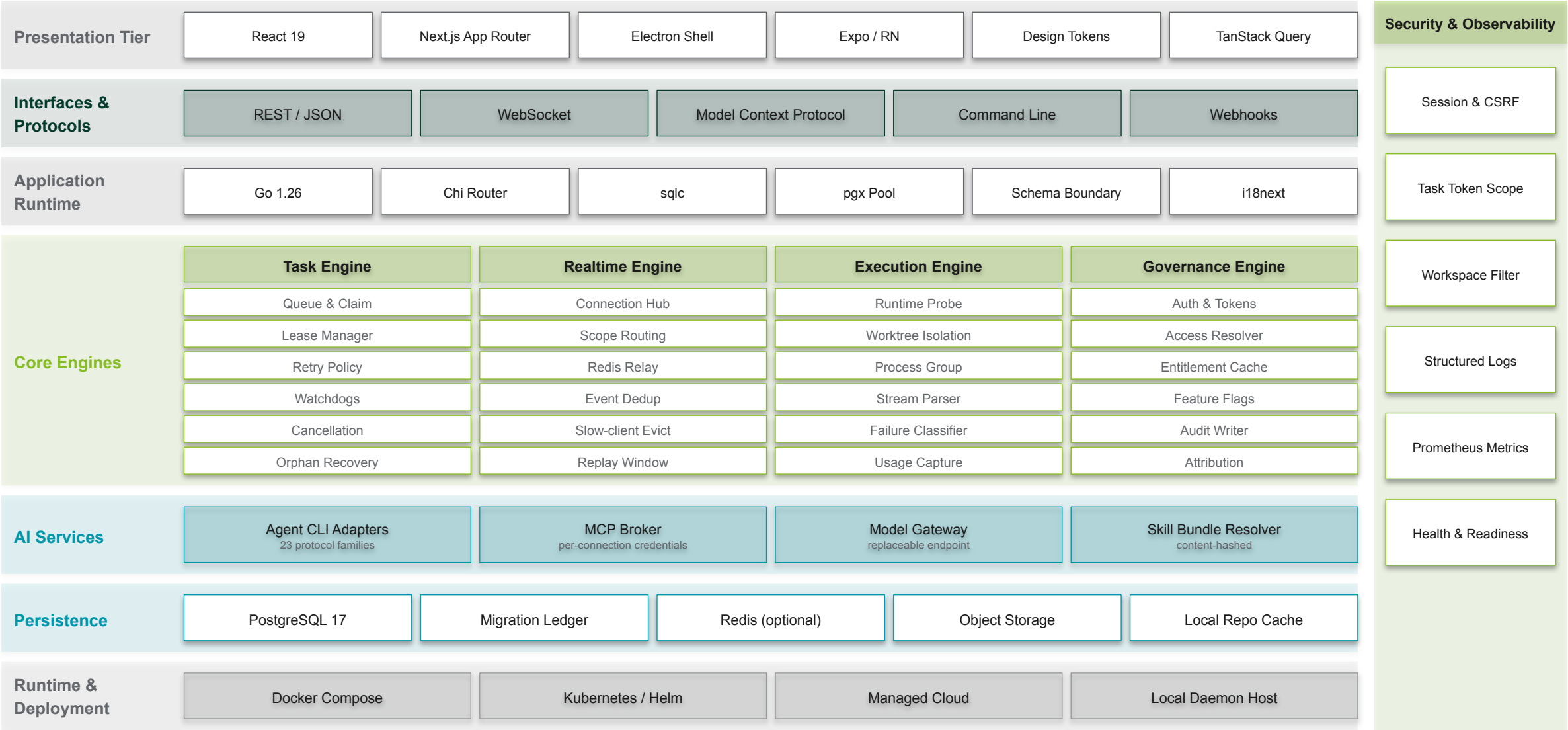
The selection rule: **if a skill can do it, do not write a plugin.** A skill is content — no code review, no grant flow. A plugin is code — more powerful, more expensive to govern.

04 / 04

TECHNOLOGY ARCHITECTURE

What it runs on, and where the boundary sits

■ Technical Architecture



Three deployment forms, one execution model

THE STACK

Backend	Go · Chi · sqlc · pgx · WebSocket
Database	PostgreSQL 17
Cache and coordination	Redis — optional, degrades cleanly
Clients	Next.js · Electron · Expo
Execution	Go daemon + agent CLI subprocesses
Observability	Structured logs · Prometheus · tracing

23 supported agent CLI protocol families — Claude Code, Codex, Copilot, Cursor, Kimi, Qwen, Grok and more.

DEPLOYMENT FORMS

Enact Cloud MANAGED
Fast start, no infrastructure. The only form where metering is active.

Docker Compose SELF-HOSTED
Single machine or small scale. Migrations run on container start.

Kubernetes SELF-HOSTED
Helm chart with backend, frontend, database, ingress, config and alerting.

Orthogonal to all three: **where execution happens** — your own machines behind the firewall, managed cloud runtimes, or both.

DATA BOUNDARY

What stays with Enact, what never leaves your machine

This boundary is **identical** on Enact Cloud and in self-hosted deployments — the security review is done once.

Stays in Enact

CONTROL PLANE

- Workspaces, work items, comments and statuses
- Agent configuration and skills
- Task status, run records and results

— — —
dispatch
results

Stays on your machine

EXECUTION PLANE

- AI coding tools and their credentials
- Code directories and local files
- Actual file changes and command execution

ONE EXCEPTION

Anything saved into an agent's custom environment variables is stored on the Enact server and passed to the runtime at execution time. Do not put secrets there that must never leave your machine.

Tiered credentials and layered authorisation

CREDENTIAL TIERS

Session	JWT in an HttpOnly cookie; CSRF on state change
Personal access token	CLI, scripts and automation; revocable
Task token	Bound to user, agent, task and workspace; expires with the run
Daemon token	Execution-machine identity, established by pairing
Plugin token	Action API calls; rotatable and revocable
Signed capability URL	Attachments and avatars, where headers cannot travel

Task tokens are denied access to billing endpoints. An agent cannot read or act on its owner’s account.

AUTHORISATION — FOUR LAYERS

Workspace membership
Every query is filtered by workspace

Organisation role
owner / admin / member govern settings

Per-resource Access
Admins can manage but cannot bypass

Principal type
Sensitive endpoints require a human principal

Enterprise identity: SSO via OIDC or SAML, SCIM provisioning, domain allowlists, and a global switch for self-service workspace creation.

Where the real boundary sits

Enact does not promise a filesystem sandbox — and says so deliberately.

A partial sandbox breaks the toolchains agents legitimately need — installing dependencies, running builds, using cloud CLIs — while still failing to stop a task from reading credentials and sending them over the network.

WHAT THE PLATFORM DOES PROVIDE

- Per-task git worktree — the agent sees your changes, never writes your directory, never discards work
- Per-task tool state and temporary directories
- Task-scoped credentials that cannot impersonate anyone
- Preparation in an isolated process group with layered watchdogs

These reduce blast radius. They are not an adversarial security boundary.

WHERE TO PUT THE BOUNDARY

A task runs with the full permissions of the operating-system user running the daemon. Build the boundary outside it.

Dedicated system user Only the repositories and credentials the agents need	BASELINE
Container Only the mounts and secrets required	MEDIUM
Virtual machine Full isolation, at the cost of a machine	STRONG

Treat every credential reachable from that environment as one the agent may use.

Optional subsystems degrade — they do not fail

IF THIS IS ABSENT	DEGRADED BEHAVIOUR	IMPACT
Redis	Single-node in-memory realtime; no rate limiting; claim caches query the database	Events do not cross nodes in a multi-replica setup
Model gateway	No session titles or quick-action suggestions	Convenience features only
Metrics address	No metrics endpoint exposed	Observability only
A channel key	That channel’s endpoints return service unavailable	That channel only
Email service	Invitation links are logged for manual distribution	Invitations need a manual step
Any online runtime	Work waits in the queue	Nothing is lost; claimed when a runtime returns

Minimal deployments work, and local faults never escalate into outages. Execution adds leases, layered watchdogs, failure classification, orphan recovery and cancellation acknowledgement.

IN SUMMARY

The layer that gets
more valuable as agents
get stronger

- Agents as accountable teammates, not tools on a panel
- A data boundary that is identical on cloud and self-hosted
- Governance the enterprise controls: gates, evidence, isolation
- Open core — internal self-hosted use needs no commercial licence